

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ

Заместитель директора
по учебной работе

_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 14

ПРОГРАММИРОВАНИЕ КЛАССОВ. ПЕРЕГРУЗКА ОПЕРАТОРОВ

ЛАБОРАТОРНАЯ РАБОТА № 14.2

**ПРОГРАММИРОВАНИЕ КЛАССОВ С ПРИМЕНЕНИЕМ ДЕКОРАТОРОВ
КЛАССОВ**

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____ от ____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Программирование классов с применением декораторов классов

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Свойства класса

Внутри класса можно создать идентификатор, через который в дальнейшем будут производиться операции получения и изменения значения какого-либо атрибута, а также его удаления. Создается такой идентификатор с помощью функции **property()**. Формат функции:

```
<Свойство> = property(<Чтение>[, <Запись>[, <Удаление>
    [, <Строка документирования>]])
```

В первых трех параметрах указывается ссылка на соответствующий метод класса. При попытке получить значение будет вызван метод, указанный в первом параметре. При операции присваивания значения будет вызван метод, указанный во втором параметре, - этот метод должен принимать один параметр. В случае удаления атрибута вызывается метод, указанный в третьем параметре. Если в качестве какого-либо параметра задано значение None, то это означает, что соответствующий метод не поддерживается. Рассмотрим свойства класса на примере:

```
class MyClass:
    def __init__(self, value):
        self.__var = value
    def get_var(self):      # Чтение
        return self.__var
    def set_var(self, value): # Запись
        self.__var = value
    def del_var(self):      # Удаление
        del self.__var
    v = property(get_var, set_var, del_var, "Строка документирования")
c = MyClass(5)
```

```
c.v = 35 # Вызывается метод set_var()
print(c.v) # Вызывается метод get_var()
del c.v # Вызывается метод del_var()
```

Результат работы приложения:

35

Python поддерживает альтернативный метод определения свойств - с помощью методов **property()**, **setter()** и **deleter()**, которые используются в декораторах. Пример их использования приведен ниже:

```
class MyClass:
    def __init__(self, value):
        self.__var = value
    @property
    def v(self): # Чтение
        return self.__var
    @v.setter
    def v(self, value): # Запись
        self.__var = value
    @v.deleter
    def v(self): # Удаление
        del self.__var
c = MyClass(5)
c.v = 35 # Чтение
print(c.v) # Запись
del c.v # Удаление
```

Результат работы приложения:

35

Имеется возможность определить абстрактное свойство - в этом случае все реализующие его методы должны быть переопределены в подклассе. Выполняется это с помощью декоратора **@abstractmethod** из модуля **abc**. Пример определения абстрактного свойства показан ниже:

```
from abc import ABCMeta, abstractmethod
class MyClass1 (metaclass=ABCMeta):
    def __init__(self, value):
        self.__var = value
    @property
    @abstractmethod
    def v(self): # Чтение
        return self.__var
    @v.setter
    @abstractmethod
    def v(self, value): # Запись
        self.__var = value
    @v.deleter
    @abstractmethod
    def v(self): # Удаление
```

```
del self.__var
```

4.2 Создание свойств

Получить доступ к атрибуту, в том числе закрытому, в Python можно с помощью **свойств**. Чтобы создать свойство, следует написать метод, который вернет значение, однако перед тем, как метод будет объявлен, надо написать декоратор **@property**. Продемонстрируем создание свойства на примере, для чего напишем метод **vozs**, с помощью которого получим доступ к закрытому атрибуту **__vozs**.

Метод **vozs** ниже предваряем декоратором **@property**, а имя свойства будет совпадать с именем метода.

```
class Student:
    def __init__(self, name, city, vozs):
        self.name = name
        self.city = city
        self._vozs = vozs # используйте приватное поле с одним
        подчеркиванием
    @property
    def vozs(self):
        return self._vozs # возвращайте приватное поле
    def message(self):
        print("Здравствуйте, я студент", self.name, "из", self.city, "возраст",
        self.vozs)
stl = Student("Сергей", "Москва", "17")
stl.message()
```

Теперь через свойство **vozs** получить доступ к значению атрибута внутри или вне объявления класса. Непосредственно в коде оператором **self.vozs** вызывается метод **vozs**, который вернет атрибут **__vozs**. Происходит обращение к методу **vozs**, которое возвратит значение **__vozs**, т. е. число 17.

4.3 Декораторы классов в Python

В языке **Python**, помимо декораторов функций, поддерживаются также декораторы классов, которые позволяют изменить поведение самих классов. В качестве параметра декоратор принимает ссылку на объект класса, поведение которого необходимо изменить, и должен возвращать ссылку на тот же класс или какой-либо другой. Пример декорирования класса показан ниже:

```
def deco(C): # Принимает объект класса
    print("Внутри декоратора") # Производит какие-то действия
    return C # Возвращает объект класса
@deco
class MyClass:
    def __init__(self, value):
        self.v = value
```

```
c = MyClass(5)
```

```
print(c.v)
```

Результат работы приложения:

Внутри декоратора

5

4.4 Декораторы функций в Python

Для того, чтобы понять, как работают декораторы, в первую очередь следует вспомнить, что функции в python являются объектами, соответственно, их можно возвращать из другой функции или передавать в качестве аргумента.

Декораторы в Python представляют функцию, которая в качестве параметра получает функцию и в качестве результата также возвращает функцию. Декораторы позволяют модифицировать выполняемую функцию, значения ее параметров и ее результат без изменения исходного кода этой функции.

Рассмотрим простейший пример:

определение функции декоратора

```
def select(input_func):
```

 def output_func(): # определяем функцию, которая будет выполняться вместо оригинальной

 print("*****") # перед выводом оригинальной функции выводим всякую звездочки

 input_func() # вызов оригинальной функции

 print("*****") # после вывода оригинальной функции выводим всякую звездочки

 return output_func # возвращаем новую функцию

определение оригинальной функции

@select # применение декоратора select

```
def hello():
```

 print("Hello METANIT.COM")

вызов оригинальной функции

```
hello()
```

Вначале определяется собственно функция декоратора, которая в данном случае называется select(). В качестве параметра декоратор получает функцию (в данном случае параметр input_func), к которой этот декоратор будет применяться:

```
def select(input_func):
```

 def output_func(): # определяем функцию, которая будет выполняться вместо оригинальной

 print("*****") # перед выводом оригинальной функции выводим всякую звездочки

 input_func() # вызов оригинальной функции

 print("*****") # после вывода оригинальной функции

выводим всякую звездочки

 return output_func # возвращаем новую функцию

Результатом декоратора в данном случае является локальная функция `output_func`, в которой вызывается входная функция `input_func`. Для простоты здесь перед и после вызова `input_func` для красоты просто выводим набор символов `"**"`.

Далее определяется стандартная функция, к которой применяется декоратор - в данном случае это функция `hello`, которая просто выводит на консоль некоторую строку:

```
@select      # применение декоратора select
def hello():
    print("Hello METANIT.COM")
```

Для применения декоратора перед определением функции указывается символ `@`, после которого идет имя декоратора. То есть в данном случае к функции `hello()` применяется декоратор `select()`.

Далее вызываем обычную функцию:

```
hello()
```

Поскольку к этой функции применяется декоратор `select`, то в результате функция `hello` передается в декоратор `select()` в качестве параметра `input_func`. И поскольку декоратор возвращает новую функцию - `output_func`, то фактически в данном случае будет выполняться именно эта функция `output_func()`

В итоге мы получим следующий консольный вывод:

```
*****
Hello METANIT.COM
*****
```

4.5 Получение параметров функции в декораторе

Декоратор может перехватывать передаваемые в функцию аргументы:

```
# определение функции декоратора
def check(input_func):
    def output_func(*args):      # через *args получаем значения параметров
    оригинальной функции
        input_func(*args)        # вызов оригинальной функции
    return output_func           # возвращаем новую функцию
# определение оригинальной функции
@check
def print_person(name, age):
    print(f"Name: {name} Age: {age}")
# вызов оригинальной функции
print_person("Tom", 38)
```

Здесь функция `print_person()` принимает два параметра: `name` (имя) и `age` (возраст). К этой функции применяется декоратор `check()`

В декораторе `check` возвращается локальная функция `output_func()`, которая принимает некоторый набор значений в виде параметра `*args` - это те значения, которые передаются в оригинальную функцию, к которой применяется декоратор. То есть в данном случае `*args` будет содержать значения параметров `name` и `age`.

```
def check(input_func):
```

```
def output_func(*args):      # через *args получаем значения параметров
    функции input_func
```

Здесь просто передаем эти значения в оригинальную функцию:

```
input_func(*args)
```

В итоге в данном получим следующий консольный вывод:

```
Name: Tom Age: 38
```

Но что, если в функцию `print_person` будет передано какое-то недопустимое значение, например, отрицательный возраст? Одним из преимуществ декораторов как раз является то, что мы можем проверить и при необходимости модифицировать значения параметров. Например:

```
# определение функции декоратора
```

```
def check(input_func):
```

```
    def output_func(*args):
```

```
        name = args[0]
```

```
        age = args[1]      # получаем значение второго параметра
```

```
        if age < 0: age = 1    # если возраст отрицательный, изменяем его
    значение на 1
```

```
        input_func(name, age) # передаем функции значения для параметров
```

```
    return output_func
```

```
# определение оригинальной функции
```

```
@check
```

```
def print_person(name, age):
```

```
    print(f'Name: {name} Age: {age}')
```

```
# вызов оригинальной функции
```

```
print_person("Tom", 38)
```

```
print_person("Bob", -5)
```

`args` фактически представляет набор значений, и, используя индексы, мы можем получить значения параметров по позиции и что-то с ними сделать. Так, здесь, если значение возраста меньше 0, то устанавливаем 1. Затем передаем эти значения в вызов функции. В итоге здесь получим следующий вывод:

```
Name: Tom Age: 38
```

```
Name: Bob Age: 1
```

4.6 Получение результата функции

Подобным образом можно получить результат функции и при необходимости изменить его:

```
# определение функции декоратора
```

```
def check(input_func):
```

```
    def output_func(*args):
```

```
        result = input_func(*args)    # передаем функции значения для
    параметров
```

```
        if result < 0: result = 0    # если результат функции меньше нуля, то
    возвращаем 0
```

```
    return result
```

```

    return output_func
# определение оригинальной функции
@check
def sum(a, b):
    return a + b
# вызов оригинальной функции
result1 = sum(10, 20)
print(result1)      # 30
result2 = sum(10, -20)
print(result2)      # 0

```

Здесь определена функция `sum()`, которая возвращает сумму чисел. В декораторе `check` проверяем результат функции и для простоты, если он меньше нуля, то возвращаем 0.

Консольный вывод программы:

```

30
0

```

4.7 Пример решения задачи на языке программирования Python

Задача. Создайте декоратор класса, который автоматически добавляет к любому классу атрибут `created = True`. Данный атрибут должен появляться у всех объектов класса после его применения. Проверьте работу декоратора на любом примере класса.

Решение задачи:

```

def add_created_attribute(cls):
    cls.created = True
    return cls
@add_created_attribute
class Student:
    def __init__(self, name):
        self.name = name
@add_created_attribute
class Product:
    def __init__(self, title):
        self.title = title
# Проверка
s = Student("Alex")
p = Product("Laptop")
print(s.created) # True
print(p.created) # True

```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте декоратор класса, который добавляет атрибут `version = "1.0"` к любому классу.

2 Создайте декоратор класса, добавляющий атрибут `author`, значение которого задаётся в декораторе.

3 Создайте декоратор класса, который добавляет метод `get_class_name()`, возвращающий имя класса.

4 Создайте декоратор класса, автоматически добавляющий атрибут `created = True`.

5 Создайте декоратор класса, который добавляет метод `info()`, выводящий все атрибуты объекта.

6 Создайте декоратор класса, добавляющий метод `to_dict()`, возвращающий атрибуты объекта в виде словаря.

7 Создайте декоратор класса, который добавляет метод `reset()`, обнуляющий все числовые атрибуты объекта.

8 Создайте декоратор класса, добавляющий атрибут `category = "default"`.

9 Создайте декоратор класса, который добавляет метод `is_empty()`, проверяющий, есть ли у объекта атрибуты.

10 Создайте декоратор класса, автоматически добавляющий метод `describe()`, возвращающий строковое описание объекта.

11 Создайте декоратор класса, который добавляет счётчик количества созданных объектов класса.

12 Создайте декоратор класса, добавляющий атрибут `status = "active"` и метод `get_status()`.

13 Создайте декоратор класса, который переопределяет метод `__repr__()` для красивого вывода объекта.

14 Создайте декоратор класса, который автоматически добавляет атрибут `id = 0` и увеличивает его значение для каждого нового созданного объекта класса.

15 Создайте декоратор класса, который добавляет метод `print_attrs()`, выводящий только те атрибуты объекта, которые не начинаются с символа `_`.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте класс `Student` со свойством `age`, не допускающим отрицательных значений.

2 Создайте класс `Product` со свойством `price`, которое не может быть меньше нуля.

3 Создайте класс Rectangle со свойствами width и height, допускающими только положительные значения.

4 Создайте класс BankAccount со свойством balance, запрещающим установку отрицательного баланса.

5 Создайте класс Employee со свойством salary, которое не может быть меньше минимального значения (например, 100).

6 Создайте класс Temperature со свойством celsius, допускающим значения от -50 до +50.

7 Создайте класс Car со свойством speed, не превышающим 200 км/ч.

8 Создайте класс Circle со свойством radius, допускающим только положительные значения.

9 Создайте класс User со свойством email, которое не может быть пустой строкой.

10 Создайте класс Book со свойством pages, принимающим только целые положительные числа.

11 Создайте класс Box со свойством volume, запрещающим отрицательные значения.

12 Создайте класс Password со свойством value, длина которого не менее 6 символов.

13 Создайте класс Counter со свойством count, не допускающим уменьшения значения.

14 Создайте класс Circle, в котором свойство radius допускает только положительные значения. При попытке присвоить нулевое или отрицательное значение должно устанавливаться значение по умолчанию (например, 1).

15 Создайте класс Account, в котором свойство username не может содержать пробелы и не должно быть пустой строкой. При нарушении условия значение не изменяется.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Приведите синтаксис создания свойств в языке программирования Python.

7.2 Охарактеризуйте свойства в языке программирования Python.

7.3 Дайте определение понятию «Декоратор класса» в языке программирования Python.

7.4 Опишите синтаксис создания декоратора класса в языке программирования Python.

Список используемых источников

- 1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.
- 2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.
- 3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.